
Security Review Report

NM-0317 Defx



NETHERMIND
SECURITY

(Oct 10, 2024)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	User Flow	4
4.2	Validator Flow	4
4.2.1	Proposing a New Set of Validators	4
4.2.2	Withdrawals	4
4.2.3	Administrative Actions	5
4.3	Locker Flow	5
5	Risk Rating Methodology	6
6	Issues	7
6.1	[High] Signature replay attack within the cancelValidatorSetUpdate(...) function	7
6.2	[Medium] Inconsistent logic for pause and unpause functionalities	8
6.3	[Medium] The pause(...) function doesn't use whenNotPaused modifier leading to potential DoS	9
6.4	[Medium] Wrong dispute period calculation within the isTransactionInDisputeWindow(...) function	9
6.5	[Low] Potential DoS of user deposits	10
6.6	[Low] Validators are not added if already in previous set in _updateValidatorSet(...)	11
6.7	[Info] Absence of verification for validators data during initialization and updates	11
6.8	[Info] Inconsistent maximum number of iterations in the ValidatorLibrary functions	12
6.9	[Info] Inconsistency in the permit(...) functions across different networks	12
6.10	[Info] Missing signature checks within recoverSigner(...) function logic	13
6.11	[Info] Support for Fee-on-transfer tokens behaviour	13
6.12	[Info] cancelValidatorSetUpdate(...) and invalidateWithdrawals(...) does not check dispute window	13
6.13	[Info] proposeValidatorSet(...) can override pendingValidatorSetUpdate	14
6.14	[Best Practices] Domain Separator is not compliant with EIP712	14
6.15	[Best Practices] invalidateWithdrawals(...) function allows to invalidate non-requested messages	14
7	Documentation Evaluation	15
8	Test Suite Evaluation	16
8.1	Compilation Output	16
8.2	Tests Output	16
9	About Nethermind	18

1 Executive Summary

This document presents the security review performed by Nethermind Security for Defx bridge contract. The Defx team has developed a bridge contract responsible for transferring assets from various chains to their proprietary Layer 1 (L1) network. Users interact with the bridge by generating a signature for a permit message, which authorizes validators to transfer tokens on their behalf and account for them on the L1 network. Validators play a crucial role in this bridge, managing all deposits and withdrawals to ensure seamless asset transfers.

This security review focuses exclusively on the smart contracts listed in Section 2 (*Audited Files*). It is important to note that the Defx L1 network and its associated off-chain systems are not covered in this review.

The audit was performed using (a) manual analysis of the codebase, (b) automated analysis tools, (c) simulation of the smart contract, and (d) creation of test cases. **Along this document, we report** fifteen (15) points of attention, where one is classified as High, three are classified as Medium, two are classified as Low, and nine are classified as Informational or Best Practice. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the compilation, tests, and automated tests. Section 9 concludes the document.

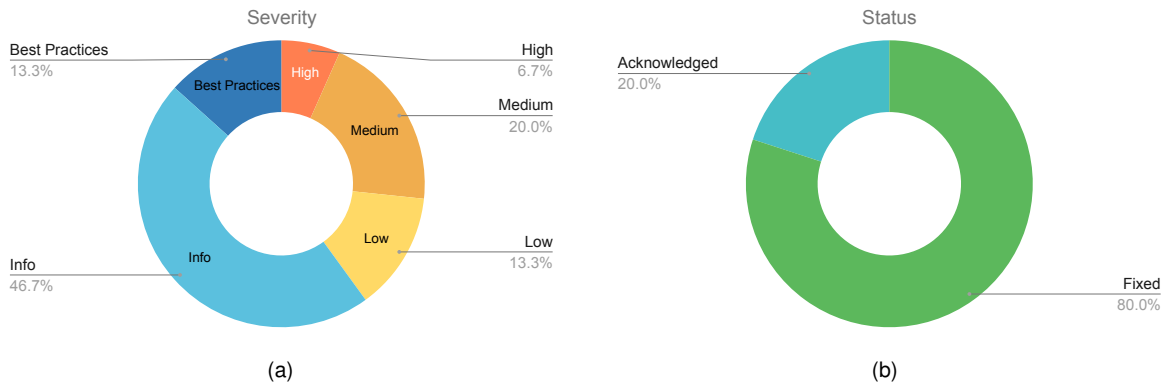


Fig. 1: Distribution of issues: Critical (0), High (1), Medium (3), Low (2), Undetermined (0), Informational (7), Best Practices (2).
Distribution of status: Fixed (12), Acknowledged (3), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	Sep 27, 2024
Response from Client	Regular responses during audit engagement
Final Report	Oct 10, 2024
Repository	defx-contract
Commit (Audit)	5b442ea6b68473f45d964ef2573c794e62a2b0ad
Commit (Final)	2eb6d520324a6dfb08347494139865402e96ac08
Documentation Assessment	Low
Test Suite Assessment	Medium

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	DefxBridge.sol	611	85	13.9%	106	802
2	common/Errors.sol	29	1	3.4%	11	41
3	common/Structs.sol	51	3	5.9%	8	62
4	common/Events.sol	44	1	2.3%	16	61
5	library/Utils.sol	38	3	7.9%	7	48
6	library/ValidatorLibrary.sol	120	4	3.3%	21	145
7	library/SignatureLibrary.sol	59	1	1.7%	8	68
	Total	952	98	10.3%	177	1227

3 Summary of Issues

	Finding	Severity	Update
1	Signature replay attack within the cancelValidatorSetUpdate(...) function	High	Fixed
2	Inconsistent logic for pause and unpause functionalities	Medium	Fixed
3	The pause(...) function doesn't use whenNotPaused modifier leading to potential DoS	Medium	Fixed
4	Wrong dispute period calculation within the isTransactionInDisputeWindow(...) function	Medium	Fixed
5	Potential DoS of user deposits	Low	Fixed
6	Validators are not added if already in previous set in _updateValidatorSet(...)	Low	Fixed
7	Absence of verification for validators data during initialization and updates	Info	Acknowledged
8	Inconsistent maximum number of iterations in the ValidatorLibrary functions	Info	Fixed
9	Inconsistency in the permit(...) functions across different networks	Info	Acknowledged
10	Missing signature checks within the recoverSigner(...) function	Info	Fixed
11	Support for Fee-on-transfer tokens behaviour	Info	Acknowledged
12	cancelValidatorSetUpdate(...) and invalidateWithdrawals(...) does not check dispute window	Info	Fixed
13	proposeValidatorSet(...) can override pendingValidatorSetUpdate	Info	Fixed
14	Domain Separator is not compliant with EIP712	Best Practices	Fixed
15	invalidateWithdrawals(...) function allows to invalidate non-requested messages	Best Practices	Fixed

4 System Overview

Defx implements a bridge allowing the transfer of assets between EVM-compatible blockchains, initially supporting Arbitrum and Defx's proprietary Layer 1 (L1) blockchain. This bridge is governed by a group of validators responsible for deposits, withdrawals, and overall bridge management.

In the following subsections, we outline the workflows for the main roles within the contract: users, validators, and lockers.

4.1 User Flow

Users interact with the bridge primarily by generating a signature for the `permit(...)` function, authorizing the bridge to transfer tokens on their behalf. Validators invoke `batchDepositWithPermit(...)` using the user's signatures to transfer the tokens. This design ensures a gasless user experience, as their sole action is to sign a permit message.

```
function batchDepositWithPermit(DepositWithPermit[] calldata deposits) external whenNotPaused nonReentrant;
```

Validators are responsible for accurately accounting for these tokens on the L1 blockchain. It is important to note that this component of the protocol has not been reviewed by the Nethermind Security team; however, we operate under the assumption that it functions correctly.

To withdraw tokens from the bridge, users must initiate withdrawal actions on the L1 blockchain. Validators will then process these withdrawal requests within the bridge contract.

4.2 Validator Flow

Validators play a critical role in managing the bridge. Each validator is assigned a specific voting power established during their initialization within the bridge. When an action is proposed, validators must sign a message containing all necessary data for that action. These signatures are provided as input when calling the corresponding function. An action is executed if it receives approval from validators representing at least two-thirds of the total voting power.

4.2.1 Proposing a New Set of Validators

The initial validator set is established during the bridge's initialization. Validators can propose changes to this set using the `proposeValidatorSet(...)` function.

```
function proposeValidatorSet(
    ValidatorUpdateRequest calldata newValidatorSet,
    Signature[] calldata signatures
) external whenNotPaused;
```

Upon successful execution, the proposed validator set is stored in the `pendingValidatorSetUpdate` variable. After a predefined period, known as the dispute window, any validator can finalize the update by calling `finalizeValidatorSetUpdate(...)`, thereby updating the active validator set.

```
function finalizeValidatorSetUpdate() external whenNotPaused;
```

During the dispute window, validators can cancel the proposed update by invoking `cancelValidatorSetUpdate(...)`.

```
function cancelValidatorSetUpdate(uint64 nonce, Signature[] calldata signatures) external whenNotPaused;
```

4.2.2 Withdrawals

Similar to proposing a new validator set, withdrawals follow a two-step process. A group of validators can request withdrawals based on actions occurring on the L1 blockchain.

```
function batchRequestWithdrawals(RequestWithdrawal[] calldata withdrawals) external whenNotPaused nonReentrant;
```

The `batchRequestWithdrawals(...)` function permits validators to submit up to ten withdrawal requests in a single transaction. Each request is recorded in the `requestedWithdrawals` mapping, using its hash as the key.

Withdrawal requests can be invalidated through the `invalidateWithdrawals(...)` function.

```
function invalidateWithdrawals(
    bytes32[] calldata messages,
    uint64 nonce,
    Signature[] calldata signatures
) external nonReentrant;
```

Once the dispute window concludes, any party can finalize the withdrawal by providing the corresponding message hash to the `finalizeWithdrawal(...)` function.

```
function finalizeWithdrawal(bytes32 message) external whenNotPaused nonReentrant;
```

Executing this function completes the withdrawal process, transferring the specified tokens from the bridge to the user indicated in the request.

4.2.3 Administrative Actions

Validators have the authority to update specific storage variables within the bridge through the following administrative actions. Each action is executed in a single step if at least two-thirds of the total voting power approve the changes:

- Update of lockers addresses:

```
function modifyLocker(address locker, bool isEnabled, uint64 nonce, Signature[] calldata signatures) external
```

- Update of supported token contracts:

```
function updateTokenContracts(address[] calldata tokenContractsUpdate, uint64 nonce, Signature[] calldata  
→ signatures) external;
```

- Update of block duration in milliseconds:

```
function changeBlockDurationMillis(uint64 newBlockDurationMillis, uint64 nonce, Signature[] calldata signatures)  
→ external;
```

- Update of withdrawal dispute period:

```
function changeWithdrawalDisputePeriodSeconds(uint64 newDisputePeriodSeconds, uint64 nonce, Signature[] calldata  
→ signatures) external;
```

- Update of validator set dispute period:

```
function changeValidatorSetDisputePeriodSeconds(uint64 newDisputePeriodSeconds, uint64 nonce, Signature[]  
→ calldata signatures) external;
```

- Change of locker threshold:

```
function changeLockerThreshold(uint64 newLockerThreshold, uint64 nonce, Signature[] calldata signatures)  
→ external;
```

4.3 Locker Flow

Lockers constitute an additional role within the bridge, responsible for pausing and unpausing bridge operations. By default, each validator also serves as a locker; however, other users may also be assigned the locker role.

When a locker initiates a pause, its address is added to an array tracking which lockers have triggered the `pause()` function. The bridge enters a paused state once the number of lockers invoking the pause meets the threshold defined by `lockerThreshold`.

```
function pause() external;
```

The unpause functionality also requires that the threshold for resuming operations be met. Importantly, only the lockers who participated in pausing the contract are considered when determining if the threshold is reached.

```
function unpause() external whenPaused;
```

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [High] Signature replay attack within the cancelValidatorSetUpdate(...) function

File(s): DefxBridge.sol

Description: The cancelValidatorSetUpdate(...) function enables the cancellation of a proposed validator set. This function takes the nonce and the validators' signatures as inputs. The signed message by the validators includes the nonce and the string "cancelValidatorSetUpdate".

However, the nonce is not incremented after the function call. This oversight facilitates a signature replay attack, allowing an attacker to cancel the proposed validator set repeatedly. If the validators have ever canceled a proposed set at least once, an attacker can exploit this by continuously submitting the same nonce and signatures to block validator updates.

```
1 function cancelValidatorSetUpdate(  
2     uint64 nonce,  
3     Signature[] calldata signatures  
4 ) external whenNotPaused {  
5     // @audit-issue nonce is not updated  
6     bytes32 messageHash = SignatureLibrary.generateUniqueMessageHash(  
7         keccak256(abi.encode("cancelValidatorSetUpdate", nonce)),  
8         address(this)  
9     );  
10    // ...  
11 }
```

Recommendation(s): Implement the _verifyAndIncrementNonce(...) function to ensure the usage of a unique nonce within the signed message.

Status: Fixed

Update from the Client:

6.2 [Medium] Inconsistent logic for pause and unpause functionalities

File(s): DefxBridge.sol

Description: The contract can be paused using the pause() function, which requires a minimum number of lockers (lockerThreshold) to vote for the action. Each time pause() is invoked, the locker address is added to the lockersVotingLock array. When the array reaches the specified threshold, the contract is paused.

To unpause the contract, lockers must call the unpause() function. When a locker invokes this function, its address is removed from the lockersVotingLock array. The contract will unpause if the length of the array falls below lockerThreshold, as illustrated in the code below. However, this mechanism does not ensure that there is a minimal number of locker votes to unpause the contract.

For instance, if lockerThreshold is set to 5, meaning that five lockers are required to pause the contract, it suffices for just one locker to call unpause() to unlock it. Upon a successful call, that locker's address is removed from the lockersVotingLock array, resulting in a length of 4. Consequently, the condition (lockersVotingLock.length < lockerThreshold) evaluates to true, leading to the contract being unpaused.

```
1  function unpause() external whenPaused {
2      // ...
3      for (uint64 i = 0; i < lockersVotingLock.length; i++) {
4          if (lockersVotingLock[i] == msg.sender) {
5              lockersVotingLock[i] = lockersVotingLock[
6                  lockersVotingLock.length - 1
7              ];
8              // @audit locked removed from the array after voting
9              lockersVotingLock.pop();
10             break;
11         }
12     }
13
14     //@audit Does not require the entire `lockersVotingLock` set to vote to unpause the contract
15     if (lockersVotingLock.length < lockerThreshold) {
16         _unpause();
17         emit ContractResumed();
18     }
19 }
```

Recommendation(s): Ensure that at least lockerThreshold lockers must vote in order to unpause the contract. This can be achieved by checking if the length of the lockersVotingLock array reaches zero. Note that this approach is effective only if the whenNotPaused modifier is added to the pause() function to prevent potential DoS attacks. Alternatively, maintain an internal count of votes to manage the unpausing process.

Status: Fixed

Update from the Client:

6.3 [Medium] The pause(...) function doesn't use whenNotPaused modifier leading to potential DoS

File(s): DefxBridge.sol

Description: The pause(...) function enables lockers to vote to pause the contract. Once the number of pause votes reaches the defined threshold, the contract is paused.

However, the function lacks the whenNotPaused modifier. As a result, it can be invoked even when the contract has already been paused. This allows additional locker votes to accumulate in the lockersVotingLock array, potentially preventing the contract from being unpaused.

```

1  function pause() external {
2      // ...
3      // @audit locker is pushed to the array
4      lockersVotingLock.push(msg.sender);
5
6      // ...
7      // @audit: Function does not revert if already paused
8      if (lockersVotingLock.length >= lockerThreshold && !paused()) {
9          _pause();
10         emit ContractPaused();
11     }
12 }

```

Recommendation: Consider adding the whenNotPaused modifier into the pause function to ensure it cannot be called when the contract is already paused.

Status: Fixed

Update from the client:

6.4 [Medium] Wrong dispute period calculation within the isTransactionInDisputeWindow(...) function

File(s): Utils.sol

Description: The isTransactionInDisputeWindow(...) function determines whether a withdrawal request or a validator set update is within the dispute window. This window allows validators to cancel the request and prevent its execution if necessary.

When withdrawal requests are initiated, the current block.timestamp is stored in the requestedTime variable associated with the withdrawal request. Upon finalization of the withdrawal, the system checks whether it is still within the dispute window by passing requestedTime as the time parameter to the isTransactionInDisputeWindow(...) function.

The time parameter is intended to be in milliseconds to ensure the correct functioning of the isTransactionInDisputeWindow(...) function. However, block.timestamp returns the time in seconds, leading to requestedTime being stored in seconds. This discrepancy in units results in an incorrect evaluation of the dispute window.

The same issue applies to the validators set update as the epochTimestamp parameter refers to a timestamp in seconds.

```

1  function isTransactionInDisputeWindow(
2      uint64 time,
3      uint64 blockNumber,
4      uint64 disputePeriodSeconds,
5      uint64 blockDurationMillis
6  ) internal view returns (bool) {
7      bool enoughTimePassed = block.timestamp * 1000 >
8          time + disputePeriodSeconds * 1000; // @audit `time` must be in milliseconds, but is currently in seconds
9      if (!enoughTimePassed) {
10         return true;
11     }
12     // ...
13 }

```

Recommendation(s): Modify the time parameter to use milliseconds consistently or adjust the calculations to use seconds instead of milliseconds throughout the function.

Status: Fixed

Update from the Client:

6.5 [Low] Potential DoS of user deposits

File(s): DefxBridge.sol

Description: The batchDepositWithPermit(...) function enables users to deposit tokens into the contract using a permit flow. This involves users signing a permit transaction to grant the DefxBridge contract an allowance for a specified amount. When the permit(...) function is invoked, this allowance is set, and the token transfer occurs immediately afterward.

However, an issue arises because the permit(...) function can be called by any user. This means a malicious actor could potentially front-run calls to batchDepositWithPermit(...) by invoking permit(...) with the same signatures. If this happens, the allowance for the contract is set, but the subsequent call to batchDepositWithPermit() fails, leading to the blocking of the token transfer.

```
1 function batchDepositWithPermit(...) external whenNotPaused nonReentrant {
2     // ...
3     for (uint256 i = 0; i < noOfDeposits; i++) {
4         // ...
5         // @audit Call to `permit(...)` can be front-run, which results in this function call reverting
6         transactionToken.permit(
7             deposits[i].user,
8             spender,
9             deposits[i].amount,
10            deposits[i].deadline,
11            deposits[i].signature.v,
12            bytes32(deposits[i].signature.r),
13            bytes32(deposits[i].signature.s)
14        );
15
16        transactionToken.safeTransferFrom(
17            deposits[i].user,
18            spender,
19            deposits[i].amount
20        );
21    }
22 }
```

Recommendation(s): Consider adding the call to permit(...) in a try-catch block. Allowing the token transfer when the contract has the necessary allowance.

Status: Fixed

Update from the client:

6.6 [Low] Validators are not added if already in previous set in _updateValidatorSet(...)

File(s): DefxBridge.sol

Description: The _updateValidatorSet function updates the lockers mapping by adding new validators and removing old ones. However, a significant issue arises due to the order of these operations. If a validator from the old set is meant to be retained in the new set, it will be unintentionally excluded because the removal of old validators occurs after new validators have been added.

```
1 function _updateValidatorSet(...) internal {
2   //..
3   //..
4   // Add new validators to lockers
5   for (uint64 i = 0; i < validatorsColdWallets.validators.length; i++) {
6     lockers[validatorsColdWallets.validators[i]] = true;
7   }
8
9   // Remove old validators from lockers
10  //@audit Validator cannot be carried over from the old set to the new set
11  // because it will be deactivated here
12  for (uint64 i = 0; i < oldValidatorsColdWallets.validators.length; i++) {
13    lockers[oldValidatorsColdWallets.validators[i]] = false;
14  }
15  //..
16  //..
17 }
```

Recommendation(s): Reverse the order of operations. First, remove the old validators from the lockers mapping, and then add the new validators.

Status: Fixed

Update from the client:

6.7 [Info] Absence of verification for validators data during initialization and updates

File(s): DefxBridge.sol

Description: There are missing input validations when initializing and updating the validator set:

- During the call to the initialize(...) function, the initial set of validators is added directly via the _updateValidatorSet(...) function. This approach bypasses the checks enforced by the validateValidatorUpdateRequest(...) function, which could lead to incorrect cumulative power calculations and impact various contract functionalities.
- Additionally, the validateValidatorUpdateRequest(...) function fails to verify that a validator does not use the same address for both cold and hot wallets.

Recommendation(s): Incorporate the validateValidatorUpdateRequest(...) function within the initialize(...) function to ensure all necessary checks are performed during initialization. Also, add a check to verify that each validator has unique addresses for their cold and hot wallets.

Status: Acknowledged

Update from the client: Since this calling is something we control & this only ever happens once, we will not be making this change. Main reason for not doing this is to make sure we don't hit the code size limit.

6.8 [Info] Inconsistent maximum number of iterations in the ValidatorLibrary functions

File(s): [ValidatorLibrary.sol](#)

Description: The function `validateValidatorUpdateRequest(...)` contains a loop that iterates through all elements of the validator set. The loop is limited to 255 iterations due to the `uint8` type used as the index variable.

```

1 function validateValidatorUpdateRequest(...) internal view {
2     // ...
3     uint256 noOfValidators = newValidatorSet.powers.length;
4     //@audit loop limited to 255 iterations
5     for (uint8 i = 0; i < noOfValidators; i++) {
6         // ...
7     }
8     // ...
9 }

```

This creates inconsistency as other functions, such as `validateAddressIsValidator(...)`, use the `uint256` type for the loop index to iterate through the validator set. It should be clearly stated whether the bridge should be managed by a maximum of 255 validators. Based on that decision, consistency in the number of possible iterations through the validator set must be maintained.

Recommendation(s): Implement a consistent number of iterations, and if there is an expected maximum amount of validators, enforce this check in the `validateValidatorUpdateRequest(...)` function.

Status: Fixed

Update from the client:

6.9 [Info] Inconsistency in the permit(...) functions across different networks

File(s): [DefxBridge.sol](#)

Description: The `batchDepositWithPermit(...)` function utilizes a `permit(...)` function that lacks consistency across various blockchain networks. This inconsistency can lead to functionality issues when interacting with certain ERC20 tokens.

For instance, consider the DAI token on Ethereum and Arbitrum networks which has different signatures:

- Ethereum Mainnet [DAI](#)

```
function permit(address holder, address spender, uint256 nonce,
    uint256 expiry, bool allowed, uint8 v, bytes32 r, bytes32 s);
```

- Arbitrum [DAI](#)

```
function permit(address owner, address spender, uint256 value,
    uint256 deadline, uint8 v, bytes32 r, bytes32 s);
```

The `batchDepositWithPermit(...)` function is currently using the Arbitrum version of `permit(...)`. As a result, this implementation is incompatible with the Ethereum Mainnet DAI, where the `permit(...)` function has a different signature. This discrepancy prevents the `batchDepositWithPermit(...)` function from operating correctly on Ethereum for DAI token. Note that other tokens can also implement the `permit(...)` function differently.

Recommendation(s): Ensure the support of the `permit(...)` signature before allowing a token to be used within the bridge. Alternatively, implement a deposit mechanism where the user can directly deposit tokens without relying on `permit(...)` function.

Status: Acknowledged

Update from the Client: We have made a note for this information. While we won't be doing any code change w.r.t this for now but will keep this in mind while releasing other tokens.

6.10 [Info] Missing signature checks within recoverSigner(...) function logic

File(s): [SignatureLibrary.sol](#)

Description: The `recoverSigner(...)` function relies on the `ecrecover(...)` function to recover the message signer. However, this function presents some issues:

Firstly, the function does not check the `s` value, allowing for signature malleability. With elliptic curves, two valid signatures exist for a given message and private key: one with a positive `s` value and one with a negative `s` value. The `SignatureLibrary::recoverSigner` function fails to enforce that the `s` value is in the lower half of the elliptic curve's order, which could theoretically enable users to create an alternative valid signature. However, this issue currently has no direct impact on the protocol due to the use of nonces.

Secondly, the `recoverSigner(...)` function does not check the length of the signature or the value of `v`. To address the signature malleability issue, the `v` value must be either the 27th or 28th byte, and the `s` value should be in the lower half of the elliptic curve's order. Additionally, the signature should typically be 65 bytes long. The code should enforce these checks to ensure proper signature formatting before parsing, as incorrect lengths can lead to erroneous address recovery.

Recommendation(s): Consider using the [ECDSA](#) library from OpenZeppelin to correctly implement the signer recovery function. Alternatively, implement the necessary checks on the signature to avoid the above mentioned issues.

Status: Fixed

Update from the client:

6.11 [Info] Support for Fee-on-transfer tokens behaviour

File(s): [DefxBridge.sol](#)

Description: The bridge supports any token that the group of validators whitelists. Some tokens may implement fee-on-transfer functionality, allowing the contract owner to deduct a fee on each transfer. As a result, users receive fewer tokens than originally sent. For example, the USDT token supports this functionality but is currently disabled.

Validators must ensure that they accurately account for token amounts on their proprietary L1 network to prevent any accounting issues, including the possibility of fee-on-transfer behaviour.

Defx relies on `Transfer` events to account for assets on L1. While USDT emits the deducted token amount in the `Transfer` event, some tokens may not emit these events correctly.

Recommendation(s): Ensure the behaviour of the fee-on-transfer token is taken into consideration when accounting tokens in the bridge.

Status: Acknowledged

Update from the client: We have noted this at our end and will keep in mind while releasing support for new tokens. For now, we will only support USDC which doesn't seem to have this problem if I understand correctly.

6.12 [Info] cancelValidatorSetUpdate(...) and invalidateWithdrawals(...) does not check dispute window

File(s): [DefxBridge.sol](#)

Description: Proposing a new set of validators and processing withdrawals involves a two-step finalization process. When the requests are initiated, they can be finalized after a designated period known as the dispute window.

During the dispute window, these requests can be invalidated to prevent malicious or erroneous updates. Specifically, the `cancelValidatorSetUpdate(...)` function handles the invalidation of validator set updates, while the `invalidateWithdrawals(...)` function manages the invalidation of withdrawal requests.

However, these functions do not verify whether the corresponding request is still within the dispute window. As a result, even after the dispute window has expired and the requests are ready to be finalized, validators can still call these functions to invalidate them. This oversight can prevent the successful finalization of legitimate validator set updates and withdrawals.

Recommendation(s): Implement a check within both the `cancelValidatorSetUpdate(...)` and `invalidateWithdrawals(...)` functions to ensure that the request is still within the dispute window before allowing it to be invalidated.

Status: Fixed

Update from the client:

6.13 [Info] proposeValidatorSet(...) can override pendingValidatorSetUpdate

File(s): [DefxBridge.sol](#)

Description: The validator set can be updated through a two-step process initiated by the `proposeValidatorSet(...)` function. When this function is called, the newly proposed validator set is stored in the `pendingValidatorSetUpdate` storage variable, and a dispute window begins. During the dispute window, it is possible to cancel the pending validator set using `cancelValidatorSetUpdate(...)`. Once the dispute window has concluded, the pending validator set can be finalized using `finalizeValidatorSetUpdate(...)`.

The current implementation of the `proposeValidatorSet(...)` function does not check if there is an existing pending validator set update. This allows validators to overwrite the `pendingValidatorSetUpdate` without first canceling or finalizing the current pending update.

Recommendation(s): Implement a lock on the `proposeValidatorSet(...)` function to prevent it from being called while there is an active `pendingValidatorSetUpdate`.

Status: Fixed

Update from the client:

6.14 [Best Practices] Domain Separator is not compliant with EIP712

File(s): [SignatureLibrary.sol](#)

Description: The `SignatureLibrary` implements the `makeDomainSeparator()` function, which returns a domain separator. According to [EIP-712](#), the domain separator is essential for distinguishing whether a signature is intended for a specific contract on a particular chain.

In the current implementation, the `VERIFYING_CONTRACT` constant is set to `address(0)` instead of the actual contract address. The bridge contract is not vulnerable to replay attacks as `address(this)` is included in the message hash. It is best practice to set `VERIFYING_CONTRACT` to the contract's address to follow the standard.

Recommendation(s): It is recommended to follow the EIP-712 standard for issuing signatures.

Status: Fixed

Update from the client:

6.15 [Best Practices] invalidateWithdrawals(...) function allows to invalidate non-requested messages

File(s): [DefxBridge.sol](#)

Description: The `invalidateWithdrawals(...)` function is designed to invalidate pending withdrawal requests that are within the dispute window. This ensures that only legitimate withdrawal requests can be contested during this period.

However, the current implementation does not verify whether a withdrawal request has been properly initiated. As a result, the function can invalidate any message, including those that have not been requested or were invalidated before. This allows the invalidation of unrequested messages, leading to potential inconsistencies.

Recommendation(s): Consider adding a verification step to the `invalidateWithdrawals(...)` function to ensure that messages to be invalidated are requested and not invalidated before.

Status: Fixed

Update from the client:

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Defx documentation

The Defx bridge contract lacks comprehensive documentation and NatSpec comments, which may impede code understanding, maintenance, and the auditing process. However, the Defx team effectively addressed concerns and questions raised by the Nethermind Security team during their regular calls. To further improve code understanding, it is recommended that the development team includes a thorough documentation and NatSpec comments.

8 Test Suite Evaluation

8.1 Compilation Output

```
> npm run compile

> defx-drona@0.0.0 compile
> npx hardhat compile

Downloading compiler 0.8.24
Generating typings for: 41 artifacts in dir: typechain-types for target: ethers-v6
Successfully generated 132 typings!
Compiled 42 Solidity files successfully (evm target: paris).
```

8.2 Tests Output

```
> npm test

> defx-drona@0.0.0 test
> npx hardhat test

DefxBridge | Config Mutation
changeBlockDurationMillis
  should change the block duration (46ms)
  should fail if the nonce is invalid
  should fail if signatures are invalid (38ms)
  should fail if the quorum is not met
changeValidatorSetDisputePeriodSeconds
  should change the dispute period
  should fail if the nonce is invalid
  should fail if signatures are invalid
  should fail if the quorum is not met
changeWithdrawalDisputePeriodSeconds
  should change the dispute period
  should fail if the nonce is invalid
  should fail if signatures are invalid
  should fail if the quorum is not met
changeLockerThreshold
  should change the locker threshold
  should fail if signatures are invalid
  should fail if the quorum is not met
  should fail if the nonce is invalid
modifyLocker
  should enable locker (203ms)
  should disable locker (63ms)
  should fail if quorum is not met (52ms)
  should fail if the nonce is invalid
updateTokenContracts
  should update token contracts when quorum is met (38ms)
  should fail if quorum is not met
  should fail if the nonce is invalid
  should fail if token is zero address
  should fail if length of contracts is zero (44ms)

DefxBridge | Deposit
batchDepositWithPermit: should deposit USDC into the contract
batchDepositWithPermit: should fail when deposits are empty
batchDepositWithPermit: should fail when deposits are more than 10
batchDepositWithPermit: should fail when user address is zero
batchDepositWithPermit: should fail when amount is zero
batchDepositWithPermit: should fail when deadline is expired
batchDepositWithPermit: should fail if deposit token is zero address
batchDepositWithPermit: should fail if token is not enabled
```

DefxBridge | Pausing

pause

- should pause the contract
- should not pause the contract **if** not enough lockers vote
- should not allow non-lockers to pause the contract
- should not allow the same locker to vote twice

unpause

- should unpause the contract
- should not allow non-lockers to unpause the contract
- should not allow unpause **if** the contract is not paused
- should not allow a locker who has not voted to pause to unpause

76 passing (18s)

DefxBridge | Validators

- proposeValidatorSet: should propose a new validator **set** (97ms)
- proposeValidatorSet: should fail when sender is not a validator (73ms)
- proposeValidatorSet: should fail with invalid timestamp (78ms)
- proposeValidatorSet: should fail when no of validators **in** hot and cold **set** are zero
- proposeValidatorSet: should fail when no of validators **in** hot and cold **set do** not match (60ms)
- proposeValidatorSet: should fail when number of validators & powers **do** not match (73ms)
- proposeValidatorSet: should fail when any of the validator address is zero (52ms)
- proposeValidatorSet: should fail when any of the validator power is zero (77ms)
- proposeValidatorSet: should fail when timestamp is earlier than 5 minutes from block **time** (72ms)
- proposeValidatorSet: should fail when timestamp is zero (73ms)
- proposeValidatorSet: should fail when validator does not have quorum (88ms)
- finalizeValidatorSetUpdate: should succeed when dispute period is over (91ms)
- finalizeValidatorSetUpdate: should fail when dispute period is not over (86ms)
- finalizeValidatorSetUpdate: should fail when sender is not a validator
- finalizeValidatorSetUpdate: should fail when pending validator **set** update is already finalized
- cancelValidatorSetUpdate: should cancel the pending validator **set** update (93ms)
- cancelValidatorSetUpdate: should fail when no pending validator **set** update is present
- cancelValidatorSetUpdate: should fail when validator does not have quorum (84ms)

DefxBridge | Withdrawal

Request Withdrawal

- validators should be able to request withdrawal
- should revert **if** the withdrawals are empty
- should revert **if** the withdrawals are more than 10
- should revert **if** the user is zero address
- should revert **if** the amount is zero
- should revert **if** the signatures are invalid
- should revert **if** there is no quorum
- should emit WithdrawalFailed with error code 1 **if** the withdrawal has been invalidated (40ms)
- should emit WithdrawalFailed with error code 2 **if** the withdrawal has already been requested
- should revert **if** withdrawal token is zero address
- should revert **if** token is not supported

Invalidate Withdrawal

- should invalidate the withdrawal

Finalize Withdrawal

- should finalize the withdrawal (65ms)
- should emit WithdrawalFailed with error code 3 **if** withdrawal has been invalidated (38ms)
- should emit WithdrawalFailed with error code 4 **if** the withdrawal has already been finalized (52ms)
- should emit WithdrawalFailed with error code 5 **if** the withdrawal has not been requested
- should emit WithdrawalFailed with error code 6 **if** the dispute period is not satisfied (41ms)

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our Cryptography Research team is dedicated to continuous internal research while fostering close collaboration with external partners. The team has expertise across a wide range of domains, including cryptography protocols, consensus design, decentralized identity, verifiable credentials, Sybil resistance, oracles, and credentials, distributed validator technology (DVT), and Zero-knowledge proofs. This diverse skill set, combined with strong collaboration between our engineering teams, enables us to deliver cutting-edge solutions to our partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

Learn more about us at nethermind.io.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.